

MODULE - 11

e-PG Pathshala

Subject: Computer Science

Paper: Computer Graphics and Visualization

Module: 3D Object Representations (Hermite and Bezier Splines)

Module No: CS/CGV/11

Quadrant 1– e-text

Objectives:

- Hermite Spline
- Bezier Spline

Discussion:

In the previous module, we had learnt about the two fundamental classes of splines, viz., Interpolating and Approximating. Let's discuss about them with one example for each. Hermite spline is an example for Interpolating splines and Bezier spline is an example for approximating splines. The type of spline preferred, depends on the end application in context. A Spline is represented using a set of boundary conditions, a matrix that corresponds to coefficients, and a set of blending functions that make up the spline curve. Recall the spline terminology (Module-10), that has the terms, control points, blending functions, parametric forms, and boundary conditions.

Hermite Spline:

The Hermite spline is a cubic Interpolating spline. It touches all the control points that are placed in an orderly manner. Between two successive control points we need to identify a cubic curve section such that it satisfies the boundary conditions. The Hermite spline curve is thus a combination of many smaller cubic curves that are connected end to end, with each end being the position of a control point.

Given a set of control points $P_i = (x_i, y_i, z_i)$, $i = 0, 1, 2, \dots, n$

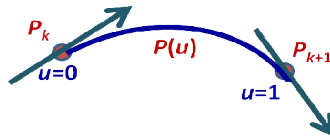
Cubic interpolation spline is obtained by fitting the input points with a piecewise cubic parametric polynomial curve that passes through every control point. A cubic parametric curve can be represented as

$$C(u) = au^3 + bu^2 + cu + d, \quad 0 \leq u \leq 1$$

where u is the parameter. The highest degree to which the parameter is raised is 3, so it is called a *cubic* curve. For a curve to be smooth enough the degree should be a minimum of 3, as quadratic curves are less smooth than the cubic ones.

We need to determine the values of a , b , c and d in the equation for the curve section. Each of these coefficients is a vector and has 3 components. For a , the components are (a_x, a_y, a_z) , and similarly the other coefficients. By setting enough *boundary conditions* at the joints between curve sections we can obtain numerical values for all the coefficients.

For two curve sections to join smoothly at the boundary, they should satisfy *zeroth* order, *first* order and *second* order parametric continuity (refer to Module-10). Between two successive control points we need to fit a cubic parametric curve.



As shown above, say there are two consecutive control points, P_k, P_{k+1} . Let $P(u)$ be the curve section that needs to be fit between the two control points. The curve should touch both P_k and P_{k+1} on either ends and should be smooth enough. The directions at either ends represent tangents to the curve section at the ends. The four boundary conditions that the curve should satisfy are given as below.

- $P(0) = P_k$ (substitute 0 for u , in the curve equation we should get the point P_k)
- $P(1) = P_{k+1}$ (substitute 1 for u , in the curve equation we should get the point P_{k+1})
- $P'(0) = DP_k$ (substitute 0 for u , in the first derivative to curve equation, we should get the tangent to the curve at the point P_k)
- $P'(1) = DP_{k+1}$ (substitute 1 for u , in the first derivative to curve equation, we should get the tangent to the curve at the point P_{k+1})

From the first condition it is evident that for $u=0$, the curve should touch one end point P_k , and from the second condition it is clear that for $u=1$, the curve should touch other end point P_{k+1} . The curve $P(u)$ can be written as a combination of a row matrix and a column matrix as shown below.

$$\begin{bmatrix} 1 & u & u^2 & u^3 \end{bmatrix} \begin{bmatrix} P_k & P_{k+1} & DP_k & DP_{k+1} \end{bmatrix}$$

The first derivative to the curve also can be represented in matrix notation as

$$\begin{bmatrix} 0 & 1 & 2u & 3u^2 \end{bmatrix} \begin{bmatrix} P_k & P_{k+1} & DP_k & DP_{k+1} \end{bmatrix}$$

The four relations / boundary conditions can be conveniently represented in matrix notation as below.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} P_k & P_{k+1} & DP_k & DP_{k+1} \end{bmatrix}$$

For ex: as per the matrix notation above $P_k = P(0) =$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} P_k & P_{k+1} & DP_k & DP_{k+1} \end{bmatrix}$$

For ex: as per the matrix notation above $P_{k+1}=P(1) =$

$$P(1) = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

For ex: as per the matrix notation above $DP_k=P'(0) =$

$$P'(0) = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

For ex: as per the matrix notation above $DP_{k+1}=P'(1) =$

$$P'(1) = \begin{bmatrix} 3 & 2 & 1 & 0 \\ 3 & 2 & 1 & 0 \\ 3 & 2 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix}$$

Thus when we combine the four conditions into a matrix form we get a 4x4 matrix as shown above. But our aim is to compute the coefficients **a**, **b**, **c** and **d**. So we can rewrite the matrix representation as below

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 3 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Where M_H is the Hermite matrix given by

$$\begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

Now substitute for the above relation in the equation for $P(u)$

$$\begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

When we multiply the three matrices, we get

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

As can be observed from the above equation, $P(u)$ is a combination of 4 cubic parametric polynomial curves, and each of these curves are called Hermite polynomials or blending functions.

$$P(u) = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 \end{bmatrix} \begin{bmatrix} H_0(u) & H_1(u) & H_2(u) & H_3(u) \end{bmatrix}$$

where

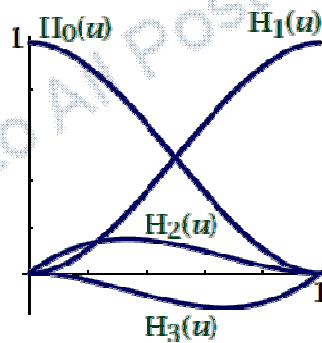
$$H_0(u) = 2u^3 - 3u^2 + 1$$

$$H_1(u) = 2u^3 - 3u^2$$

$$H_2(u) = 2u^3 - 2u^2$$

$$H_3(u) = 2u^3 - 2u^2$$

The four Hermite polynomials are called blending functions, and because for every value of the parameter in the interval 0 to 1, the four polynomials blend in some proportion to compute a point on the curve. To test whether the curve touches the two end points substitute for u , 0 in the equation for $P(u)$ and we get P_k since the coefficient associated with P_k evaluates to 1 for $u=0$, and all other blending functions evaluate to 0 for $u=0$. These curves also satisfy a minimum of C^1 continuity at the boundary.



If the shapes of the Hermite polynomials are observed carefully, it can be inferred that $H_0(u)$ has max. value of 1 at $u=0$ and min. value of 0, at $u=1$, while $H_1(u)$ has a max. value of 1 at $u=1$ and min. value of 0 at $u=0$, and $H_2(u)$ has max. and min. values somewhere between 0 and 1 for all values of u in $[0,1]$. $H_3(u)$ being the exception, where the curve is on the i ve side while all others are on the positive side.

Properties of Hermite curves:

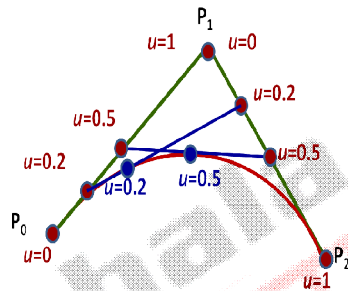
- Hermite polynomials can be adjusted *locally* because each curve section is only dependent on its end point constraints.
- One difficulty is we also need to supply the curve slopes for deriving coefficients.

Bezier Spline:

Unlike Hermite splines, Bezier splines are approximating splines, i.e., they do not touch all the control points, except for the first and last control points. If we assume a series of control points positioned in space, the curve starts with, touching the first control point, approximates all other in-between points and finally ends by touching the last control point. Bezier curves are based on Decasteljau procedure as described below.

Decasteljau algorithm: Given a set of control points placed in a sequence but positioned randomly, how can we form a curve that approximates them?

Consider a sequence of three control points P_0 , P_1 , P_2 which are *non-collinear*. Start with a value of u .



Connect the 3 non-collinear control points with two straight lines. The sequence in which we process the points is important, i.e. a line say L_{01} connects P_0 and P_1 while the other line L_{12} connects P_1 and P_2 . Assume parametric form for lines L_{01} and L_{12} (slope-intercept form, slope point form are the standard types of representing lines, the third form is the parametric form with u as the parameter ranging in the interval $[0,1]$. For $u=0$, we are at one end point of the line and for $u=1$, we are at the other end point of the line. For ex: Assume a line with two end points $P_0(x_0, y_0)$ and $P_1(x_1, y_1)$, the parametric form is given as, $x = (1-u)x_0 + ux_1$, $y = (1-u)y_0 + uy_1$. Consider the parametric forms for L_{01} and L_{12} as below.

$$L_{01}(u) = (1-u)P_0 + uP_1$$

$$L_{12}(u) = (1-u)P_1 + uP_2$$

For $u=0$, we are at P_0 , on line L_{01} , and for the same u , on line L_{12} we are at P_1 . For $u=1$, we are at P_1 , on line L_{01} , and for the same u , on line L_{12} , we are at P_2 .

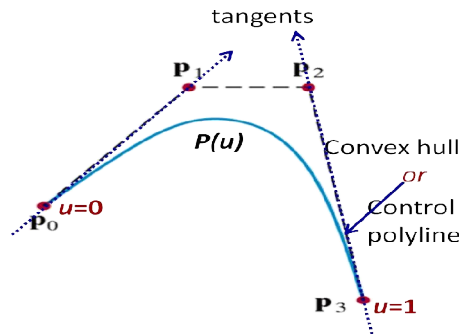
The idea is to choose a value for u (say $u=0.2$) in the interval $[0, 1]$, fix the value and find a point on the line L_{01} for the value of u . Now for the same value of u , find a point on the line L_{12} that starts from P_1 and ends at P_2 . Connect the two computed points on both the lines with another straight line, and again use the value of u to compute a point on the newly formed line (connecting points on L_{01} and L_{12}). The point that falls on the line connecting L_{01} and L_{12} shall be the point on the curve $P(u)$. If we continue this procedure for all values of u in $[0,1]$, we get a smooth flowing curve as shown in the diagram below.

The curve $P(u)$ is given by

$$P(u) = (1-u)^2 P_0 + 2u(1-u)P_1 + u^2 P_2$$

Substituting for $L_{01}(u)$ and $L_{12}(u)$ in the equation for $P(u)$ we get

$$P(u) = (1-u)^2 [(1-u)P_0 + uP_1] + 2u(1-u)[(1-u)P_1 + uP_2] + u^2 [(1-u)P_1 + uP_2]$$



$$P(u) = \sum_{k=0}^3 B_{k,3}(u) P_k$$

No. of Control points = 4 (from P_0 to P_3)

Degree of polynomials = 3 (cubic)

No. of terms in the curve equation = 4

No. of blending functions = 4

The four blending functions are $B_{0,3}(u)$, $B_{1,3}(u)$, $B_{2,3}(u)$, $B_{3,3}(u)$, each of which can be represented as $BEZ_{k,3}$, (where k varies from 0 to 3 in increments of 1). For a given value of u , the four blending functions are evaluated to obtain numerical values, called *weights*. As can be observed from the curve equation, each of the control points is associated with a specific blending function, which contributes a *weight* for a given value of u . Thus the curve is considered as a weighted combination of control points. The curve touches the first control point P_0 and the last control point P_3 but approximates P_1 and P_2 . The tangents to the curve at the first and the last control points are along the direction of the control polyline connecting, P_0 to P_1 , and P_2 to P_3 respectively. The Bezier curve equation with 4 control points can be written in general form as

$$P(u) = \sum_{k=0}^3 B_{k,3}(u) P_k, \quad u \in [0,1]$$

The four blending functions are called as Bernstein Polynomials.

$$\begin{aligned} B_{0,3}(u) &= (1-u)^3 \\ B_{1,3}(u) &= 3u(1-u)^2 \\ B_{2,3}(u) &= 3u^2(1-u) \\ B_{3,3}(u) &= u^3 \end{aligned}$$

If we extend the Bezier curves to n control points, the equation for the curve is as follows

$$P(u) = \sum_{k=0}^n B_{k,n}(u) P_k, \quad u \in [0,1]$$

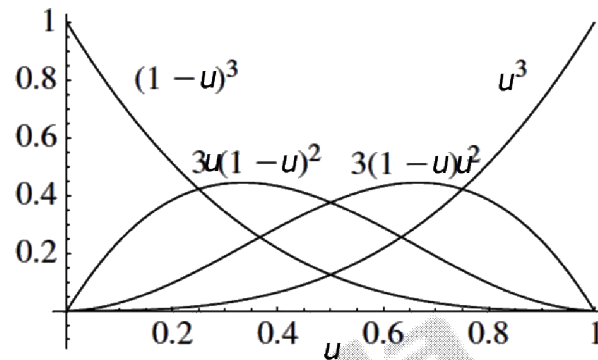
It is evident that each control point is associated with one blending function, i.e., there as many blending functions as the number of control points. The shapes of the 4 Bernstein polynomials are shown below for values of u in the interval $[0,1]$. The more the number of control points, the higher is the degree of the polynomials. A Quadratic curve has degree 2, and the number of control points being 3, while a cubic Bezier curve has the

polynomials of degree 3 with the number of control points being 4. Thus we can generalize that the degree of the polynomials is proportional to the number of control points and

$$\text{Degree} = \text{no. of control points} - 1$$

The 4 polynomial functions can be conveniently represented in the *binomial expansion* form as below.

$$BEZ_{k,3}(u) = 3C_k(1-u)^{3-k}u^k \quad k = 0,1,2,3$$



When we carefully observe the shapes of the polynomials, it is interesting to note that $BEZ_{0,3}(u)$ is maximum at $u=0$ and minimum at $u=1$, while $BEZ_{3,3}(u)$ is minimum at $u=0$ and maximum at $u=1$. The other two curves, $BEZ_{1,3}(u)$ and $BEZ_{2,3}(u)$ have their maximum at $u=1/3$ and $u=2/3$ respectively. Moreover, all the curves are on the positive side of the axis unlike the Hermite polynomials where one of the curves is on the negative side and all the 4 curves have spans across the whole of the interval $[0,1]$ of the parameter u .

Summary:

- We have learnt how Hermite Interpolating spline is derived
- Decasteljau algorithm for drawing approximate curves is discussed.
- Drawing Bezier curves by extending Decasteljau algorithm is discussed.